

ELEC 442

Lab 2

LEARNING GOALS

Inverse Kinematics (IK)

- **Handle Multi-Configuration Solutions:** Update an analytical IK solver to evaluate multiple valid joint configurations and programmatically select the optimal path based on tracking history.
- **Implement Numerical Solvers:** Build an iterative, Jacobian-based numerical root-finding solver (Newton-Raphson) for joint trajectory generation.
- **Analyze Convergence & Edge Cases:** Compare geometric vs. numerical methods regarding runtime performance, sensitivity to initial tracking states (ϕ_{prev}), and behavior near workspace boundaries.
- **Identify Kinematic Singularities:** Observe firsthand how approaching a singularity causes a standard numerical inverse Jacobian to fail or blow up.

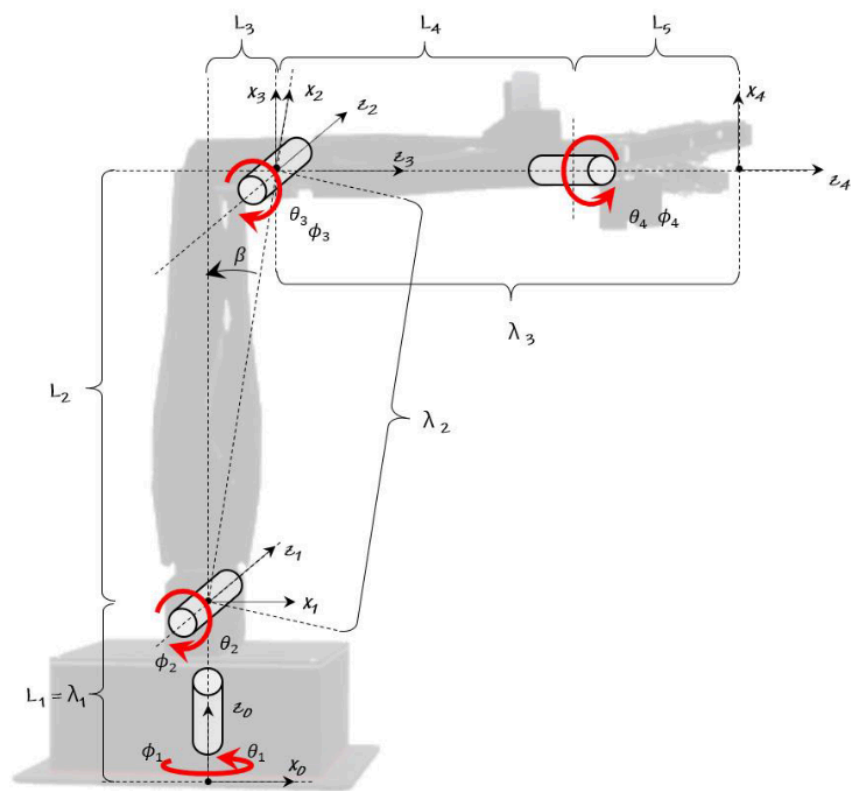
Path Planning & Obstacle Avoidance

- **Build RRT Planners:** Implement a Rapidly-exploring Random Tree algorithm in joint space utilizing custom tree-management and collision-checking classes.
- **Tune Goal Biasing:** Configure and balance random sampling thresholds vs. goal-directed steps to optimize tree growth speed.
- **Implement Potential Fields:** Map 3D Cartesian forces acting on an end-effector into joint-space steering torques using the Jacobian Transpose matrix.
- **Troubleshoot Local Minima:** Identify structural failure modes where competing attractive and repulsive vectors trap the manipulator in dead zones, and propose algorithmic solutions to escape them.

1 Concept Review

1.1 Review: The Quanser QArm

Welcome back! This second lab focuses on inverse kinematics and continues into autonomous path planning. Before diving in, let's review the physical dimensions and joint limits of the QArm manipulator.



Parameter	Value	Parameter	Value
L_1	0.14 m	L_4	0.25 m
L_2	0.35 m	L_5	0.15 m
L_3	0.05 m	β	8.13°

Figure 1: Quanser QArm Dimensions

Joint Specifications									
#	Item		Base	Shoulder	Elbow	Wrist	Units	Gripper	Units
1	Joint range	Min. Max	-170 +170	-85 +85	-95 +75	-160 +160	degrees	0 100	%
2	Max. speed		$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	rad/s	± 100	%/s
3	Max. acceleration		$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	rad/s/s	± 400	%/s/s
4	Current	Instantaneous Continuous	4.4 1.1	8.8 2.2	4.4 1.1	2.3 0.6	Amps	1.4 0.35	Amps
5	Torque	Instantaneous Continuous	10.6 2.65	21.2 5.3	10.6 2.65	4.1 1.0	Nm	1.9 0.5	Nm

Figure 2: Quanser QArm Joint limits

1.2 Inverse Kinematics Theory

When controlling a robot, we are generally able to measure and define a target destination's coordinates within a fixed task-space reference frame (x, y, z) . The fundamental challenge is commanding our manipulator, with its specific arrangement of joints and links, to reach that target. Inverse Kinematics (IK) performs the inverse mapping: calculating the configuration-space joint values ($\underline{\varphi}$) required to achieve a desired task-space pose.

There are two primary ways to approach this problem:

- **The Geometric Approach:** Given the relatively straightforward geometry of our arm, an analytical or geometric approach is highly effective. This allows us to derive algebraic equations that yield deterministic, closed-form solutions.
- **The Numerical Approach:** Alternatively, we can use an iterative numerical approach. This method leverages root-finding techniques (like the Newton-Raphson method) to perform gradient-based steps using the Jacobian matrix you explored in Lab 1. While the numerical approach can be less predictable and computationally heavier, it is far more generalizable and becomes essential when an analytical solution is difficult or impossible to derive.

1.2.1 Geometric Method

1. Setup and Intermediary Variables:

Given the end-effector position $p = (p_x, p_y, p_z)$ and link lengths $\lambda_1, \lambda_2, \lambda_3$:

$$\begin{aligned} A &= \lambda_2 \\ C &= -\lambda_3 \\ H &= \lambda_1 - p_z \\ D &= \pm \sqrt{p_x^2 + p_y^2} \end{aligned}$$

2. Joint 3 (θ_3):

$$F = \frac{D^2 + H^2 - A^2 - C^2}{2A}$$

$$\theta_3 = 2 \arctan2(C \pm \sqrt{C^2 - F^2}, F)$$

3. Joint 2 (θ_2):

$$\begin{aligned} M &= A + C \sin(\theta_3) \\ N &= -C \cos(\theta_3) \\ \cos(\theta_2) &= \frac{D \cdot M + H \cdot N}{M^2 + N^2} \\ \sin(\theta_2) &= \frac{H - N \cos(\theta_2)}{M} \\ \theta_2 &= \arctan2(\sin(\theta_2), \cos(\theta_2)) \end{aligned}$$

4. Joint 1 (θ_1):

Let the planar projection K be defined as:

$$\begin{aligned} K &= \lambda_2 \cos(\theta_2) - \lambda_3 \sin(\theta_2 + \theta_3) \\ \theta_1 &= \arctan2\left(\frac{p_y}{K}, \frac{p_x}{K}\right) \end{aligned}$$

5. Final Mapping to QArm Joints (φ):

$$\begin{aligned} \varphi_1 &= \theta_1 \\ \varphi_2 &= \theta_2 - \beta + \frac{\pi}{2} \\ \varphi_3 &= \theta_3 + \beta \\ \varphi_4 &= \gamma \end{aligned}$$

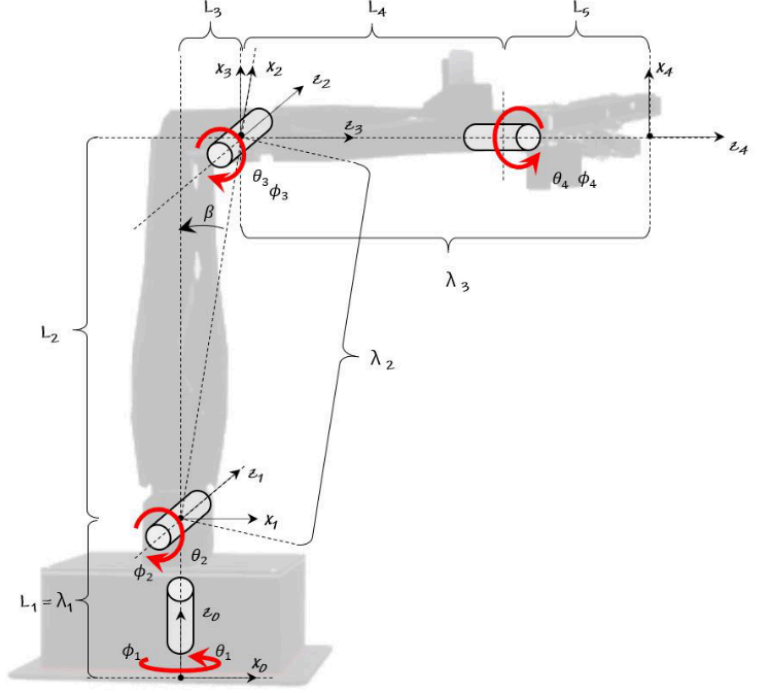


Figure 3: QArm Variables Reference

While the geometric approach is conceptually straightforward, deriving the actual analytical expressions results in a fairly bulky set of equations. Note that because the geometric approach involves solving trigonometric equations with square roots, both Joint 2 and Joint 3 yield two possible valid angles (e.g., elbow-up and elbow-down configurations). This results in a total of four distinct kinematic solutions (**allPhi**) for a given end-effector position.

1.2.2 Numerical Method (Newton-Raphson)

An alternative approach is to use an iterative numerical method. This is essentially a gradient-descent-style root-finding algorithm, where we leverage our ability to calculate the Jacobian to take successive steps from our current joint configuration toward the target position.

In its most general form, the update equation is expressed as:

$$\varphi^{i+1} = \varphi^i + \underline{J}^\dagger(\varphi^i) \begin{pmatrix} \mathcal{Q}_d - \mathcal{Q}_4(\varphi^i) \\ \underline{\theta}(\varphi^i) \underline{s}(\varphi^i) \end{pmatrix}$$

For your implementation, you will be using the analytical Jacobian. Because the QArm's three positional degrees of freedom position the wrist, and the wrist itself only features a single rotational axis (wrist roll), the orientation tracking simplifies significantly. The full 6-DOF orientation error collapses into a single

scalar value representing the wrist angle. Consequently, the analytical Jacobian simplifies to a 4×4 matrix (3 rows for x, y, z translation, and 1 row for the wrist rotation). Since this matrix is square, you can invert the Jacobian directly. Adding a scalar step size scaling factor (α), the final update equation is:

$$\varphi^{i+1} = \varphi^i + \alpha \left[J^{-1}(\varphi^i) \begin{pmatrix} \mathcal{Q}_d - \mathcal{Q}_4(\varphi^i) \\ \theta(\varphi^i) \end{pmatrix} \right]$$

Where:

- φ is the (changing) vector of joint angles
- α is the step size scaling value
- J^{-1} is the inverse Jacobian
- $\begin{pmatrix} \mathcal{Q}_d - \mathcal{Q}_4(\varphi^i) \\ \theta(\varphi^i) \end{pmatrix}$ is the 4×1 error vector (3 rows for x, y, z position and the last row for wrist angle)

i Note: Be aware that if the robot approaches a kinematic singularity, the determinant of the Jacobian approaches zero and the inverse Jacobian will spike toward infinity.

1.3 Path Planning Theory - Obstacle Avoidance

In Lab 1, you implemented a cubic spline trajectory generator using differential kinematics. While this approach works well in an unobstructed environment, real-world robotic workspaces contain physical obstacles. Moving blindly along a straight Cartesian trajectory can cause collisions with workspace objects or drive the manipulator into kinematic singularities.

To ensure safe navigation, we must proactively plan collision-free paths. This lab explores two fundamental path-planning paradigms: **Rapidly-exploring Random Trees (RRT)** and **Artificial Potential Fields (APF)**.

1.3.1 Rapidly-exploring Random Tree (RRT)

Rather than solving complex analytical path equations, RRT relies on an incremental sampling approach (LaValle, 1998). The algorithm builds a tree structure by taking small, discrete steps through the environment, verifying at each step that the configuration is collision-free before appending it to the tree. Because it randomly samples the space, RRT is “probabilistically complete”: as long as a valid path exists, the algorithm is guaranteed to find it eventually, though the resulting path is rarely the most optimal or shortest solution.

The planning process proceeds as follows:

1. **Initialization:** We begin with a known initial configuration and a target destination, both defined in joint space. The tree is initialized with the starting configuration as the root node.
2. **Random Sampling:** In each iteration, we choose a random coordinate within allowable joint boundaries.
3. **Goal Biasing:** To prevent the tree from wandering aimlessly and to bias growth toward the goal, we introduce a goal-biasing probability (e.g., a 10% chance that we skip random sampling and pick the goal directly).
4. **Extension:** We locate the existing node in our tree that is closest to our sampled target point. We then take a small step of fixed distance from that closest node toward the target.
5. **Collision Checking:** If the path segment spanning this step does not cause a collision, the new point is added to the tree as a child of the closest node.

Once a newly generated node falls within an acceptable threshold distance of the destination, you trace the path backward through parent pointers all the way to the root. Reversing this sequence yields a continuous, collision-free joint-space path.

1.3.2 Artificial Potential Fields (APF)

The Artificial Potential Fields method operates on the virtual force framework (Khatib, 1986). In this technique, we treat the robot’s workspace as an energy landscape dictated by virtual potential fields. The

target destination acts as a global minimum, generating an attractive potential that pulls the robot inward. Conversely, obstacles act as high-energy peaks, generating a repulsive potential that pushes the robot away.

To keep this exercise manageable, two simplifications are introduced:

- **End-Effector (EE) Control Only:** Your implementation will calculate the virtual forces acting strictly on the end-effector.
- **Disregarding Wrist Orientation:** Because the wrist joint only controls orientation and does not alter 3D Cartesian position, we ignore its contribution during positional force evaluation.

While Khatib originally related virtual forces ($\underline{F}$) to joint torques via $\underline{\tau} = \mathbf{J}^T \underline{F}$, we use velocity kinematics to compute instantaneous joint velocities via the inverse Jacobian:

$$\underline{\dot{q}} = \mathbf{J}^{-1} \underline{F}$$

The fictitious force formulas (adapted from Khatib, 1986) are defined as:

$$\underline{F}_{\text{attractive}} = \begin{cases} -\zeta(\underline{p}_4 - \underline{x}_{\text{goal}}) \cdot \frac{\underline{p}_4 - \underline{x}_{\text{goal}}}{\|\underline{p}_4 - \underline{x}_{\text{goal}}\|} & \text{if } \|\underline{p}_4 - \underline{x}_{\text{goal}}\| \leq d_0 \\ -\zeta d_0 \cdot \frac{\underline{p}_4 - \underline{x}_{\text{goal}}}{\|\underline{p}_4 - \underline{x}_{\text{goal}}\|} & \text{if } \|\underline{p}_4 - \underline{x}_{\text{goal}}\| > d_0 \end{cases} \quad \text{where } \underline{p}_4 \text{ is the EE position in 3D}$$

$$\underline{F}_{\text{repulsive}} = \begin{cases} \eta \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \cdot \hat{u} & \text{if } \rho \leq \rho_0 \\ \underline{0} & \text{if } \rho > \rho_0 \end{cases} \quad \text{where } \rho \text{ is distance from obstacle to EE}$$

$$\underline{F}_{\text{total}} = \underline{F}_{\text{attractive}} + \underline{F}_{\text{repulsive}}$$

$$\underline{F} = \min(\|\underline{F}_{\text{total}}\|, \alpha) \hat{\underline{F}}_{\text{total}}$$

Where:

- ζ and η are attractive and repulsive scaling gains.
- d_0 and ρ_0 are influence distance thresholds.
- $\hat{u}$ is the unit vector pointing from obstacle to the end-effector.
- α is the maximum allowable fictitious force magnitude (limiting maximum velocity/step size).

Once we have our joint space velocity ($\underline{\dot{q}}$), we update our trajectory:

$$\varphi_{\text{new}} = \varphi_{\text{prev}} + \frac{\underline{\dot{q}}}{\|\underline{\dot{q}}\|}$$

2 Pre-lab

Welcome to Lab 2! To maximize your time working with the physical robot and tuning obstacle avoidance algorithms, the Inverse Kinematics solvers should be implemented and tested **before** arriving at your lab session.

2.1 Pre-lab Setup

1. Log in to GitHub and create your group repository from the template: [Lab 2 GitHub Repository](#)
2. Clone your repository to your personal computer (and onto the lab laptop during lab time) and open the folder in VS Code.
3. Ensure your Python virtual environment and QLABs digital twin are operational.

2.1.1 Pre-lab Setup Checklist

- [] Cloned the Lab 2 repository from GitHub template.
- [] Reviewed the Geometric and Numerical IK formulas in the Concept Review.
- [] Completed Exercises 1, 2, and 3 below.

2.2 Pre-lab Exercises

2.2.1 Geometric Inverse Kinematics

To save you from tedious derivation, the geometric equations have already been implemented in `geometric_inv_kin()`. At the moment, it calculates all four possible mathematical solutions and returns `allPhi`.

```
allPhi, bestPhi = geometric_inv_kin(position, wrist, phi_prev)
```

Role	Name	Dimension	Description
input	<code>phi_prev</code>	4	Joint angles in order from base to wrist
	<code>position</code>	3	The desired position of the {4} origin in terms of the {0} coordinate system
	<code>wrist_angle</code>	1	The desired angle of the wrist joint in radians
output	<code>allPhi</code>	4×4	Matrix representing all 4 joint space solutions, each as a column
	<code>bestPhi</code>	4	The closest valid joint space solution given <code>phi_prev</code> .

Exercise 1:

At the moment, in `geometric_IK.py`, `geometric_inv_kin()` returns only `allPhi`. Edit `geometric_inv_kin()` so that we can pass in an array of joint positions as an argument (`phi_prev`), then the closest valid solution is outputted as `bestPhi`.

Description	Position (x, y, z)	wrist	phi_prev	allPhi	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ 0.87 & 0.87 & 0.87 & 0.87 \end{pmatrix}$	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ -2.79 & -2.79 & -2.79 & -2.79 \end{pmatrix}$	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} 0.79 & 0.79 & -2.36 & -2.36 \\ -1.53 & 0.14 & -0.42 & 1.24 \\ 0.12 & -2.98 & 0.12 & -2.98 \\ -2.62 & -2.62 & -2.62 & -2.62 \end{pmatrix}$	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	$\begin{pmatrix} -3.14 & 0. & -3.14 & 0. \\ -0.82 & 0.53 & -0.82 & 0.53 \\ -0.17 & -2.69 & -0.17 & -2.69 \\ 0. & 0. & 0. & 0. \end{pmatrix}$	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	$\begin{pmatrix} 1.57 & 1.57 & -1.57 & -1.57 \\ -1.79 & -0.28 & 0.00 & 1.51 \\ -0.03 & -2.83 & -0.03 & -2.83 \\ 0.00 & 0.00 & 0.00 & 0.00 \end{pmatrix}$	$\begin{pmatrix} -1.57 \\ 0 \\ -0.03 \\ 0 \end{pmatrix}$

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`.

2.2.2 Numerical Inverse Kinematics

Implement the Newton-Raphson iterative numerical solver in `numerical_IK.py`.

```
bestPhi = numeric_inv_kin(position, wrist, phi_prev)
```

Role	Name	Dimension	Description
input	phi_prev	4	Joint angles in order from base to wrist
	position	3	The desired position of the {4} origin in terms of the {0} coordinate system
	wrist_angle	1	The desired angle of the wrist joint in radians
output	bestPhi	4	The converged valid joint space solution given phi_prev.

Exercise 2: In `numerical_IK.py`, implement a numerical solver for the QArm using the provided `Inv_Jacobian()` function to get the inverse Jacobian. Several useful helper functions are listed in the python file as comments. Use the same test cases as before.

Description	Position (x, y, z)	wrist	phi_prev	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	Fail

Note: Since the numerical solver stops at a threshold, your values will be slightly different than the exact values provided by the geometric approach.

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`.

2.2.3 Inverse Kinematics Concept Check

Exercise 3:

Answer the following conceptual questions on Inverse Kinematics:

1. Why does the analytical geometric approach produce up to four distinct solutions for a given (x, y, z) position, and why is `phi_prev` necessary to select the `bestPhi` for continuous robot motion?
2. In your numerical test cases, why did the solver fail to converge at $[0, 0, 0.75]$ and $[0, -0.45, 0.5]$? What happens to the Jacobian inverse $\mathbf{J}^{-1}$ near kinematic singularities?

3 In-Lab Hardware Setup

3.1 Safety Warnings

The QArm's workspace limit will be enforced to protect the robot, surrounding equipment, and users. It will automatically stop moving if the limit is violated; however, just in case, please always keep an eye on the QArm during motion. In an emergency, keep your distance and, if safe, be prepared to turn off the power switch or terminate the program.

3.2 Hardware (Physical QArm) Setup

Please handle the QArm with care:

1. Ensure that the baseplate is on a stable, flat surface and that the QArm is mounted securely to the baseplate with the 4 corner thumb-screws.
2. Connect the QArm to the lab laptop by inserting the pair of USB-A cables into the laptop.
3. Power on the QArm.
4. When launching any of the provided lab Python programs, select your hardware target (0 for Virtual QArm, 1 for Real QArm).

3.3 Obstacle Setup

Congrats on your good work on the "Chemical Vapor Barrier" exhibit! As the only team members familiar with operating the QArm, you've now become indispensable. Your next assignment is to calibrate the QArm inside the museum's experimental metaphysics vault. The room contains massive, floor-to-ceiling cylindrical columns that look like cardboard, but they are actually the literal load-bearing pillars of reality.

If any part of the robotic arm touches these cylinders, all of reality will instantly collapse. Since you can't just move in a straight line anymore, you'll need to implement autonomous path planning to navigate the workspace safely.

Don't touch the pillars. That would be very bad.

Two obstacle setups will be used for both approaches. All start points and obstacle dimensions are defined in the provided template scripts (`obstacle_avoidance_potential.py` and `obstacle_avoidance_RRT.py`).

[Click here to view the obstacle models in 3D \(Onshape\)](#)

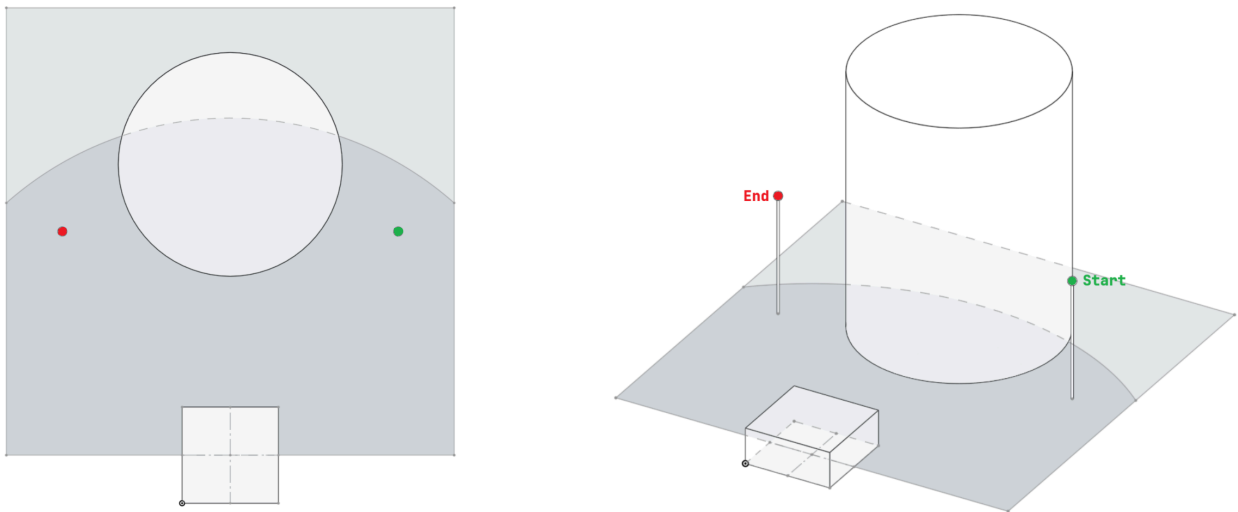


Figure 4: Obstacle Setup 1

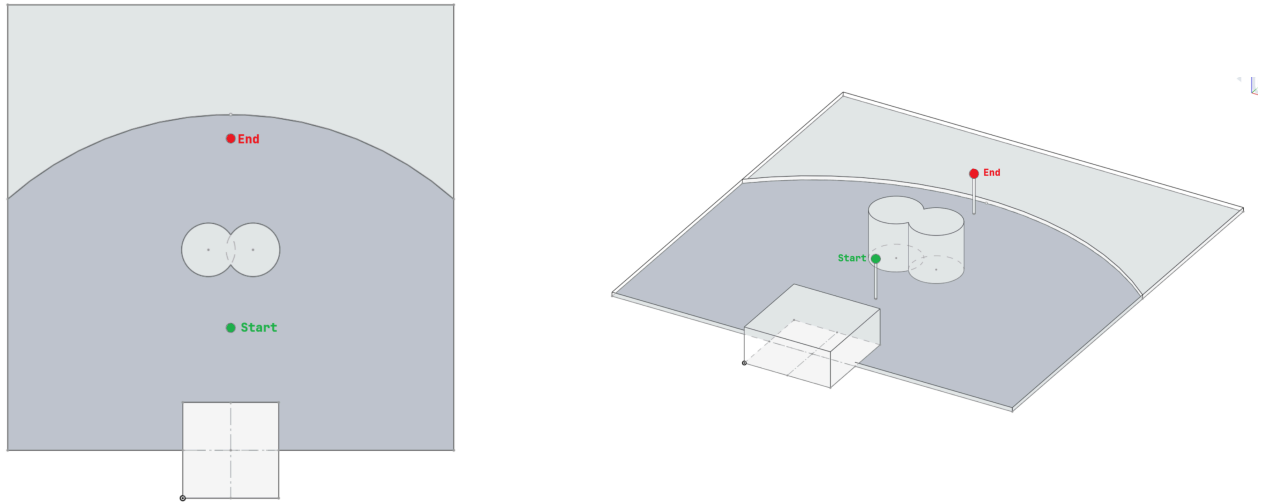


Figure 5: Obstacle Setup 2

3.4 Software Interface Reference

A key component you will interact with is the `QArm_Lab_Interface` class, detailed below:

Method	Description	Inputs	Outputs
<code>write_to_arm(phi)</code>	Writes joint commands to the arm (in radians).	<code>phi</code>	-
<code>read_from_arm()</code>	Reads the current joint positions from the arm (in radians).	-	<code>phi</code>
<code>forward_kinematics(phi)</code>	Computes forward kinematics.	<code>phi</code>	<code>p4</code> (vector from base to EE in $\{0\}$ frame), <code>R4</code> (rotation matrix from $\{4\}$ to $\{0\}$ frame)
<code>Jacobian(phi)</code>	Computes the analytic 4×4 Jacobian.	<code>phi</code>	<code>J</code> (Jacobian matrix)
<code>Inv_Jacobian(phi)</code>	Returns the inverse of the analytic 4×4 Jacobian.	<code>phi</code>	<code>J_Inv</code> (Jacobian matrix inverse)
<code>inverse_kinematics(p, gamma, phi_prev)</code>	Computes inverse kinematics.	<code>p</code> (EE 3D coords), <code>gamma</code> (wrist angle), <code>phi_prev</code>	<code>phi</code> (all 4 possible solns.), <code>phiOptimal</code> (optimal soln.)

4 In-Lab Path Planning Experiments

4.1 Rapidly-exploring Random Tree (RRT)

To help you manage tree growth, the `RRT_Tree_Manager` class is provided in your repository:

Methods	Description	Inputs	Outputs
<code>add_node(q_new, parent_index)</code>	Adds a node to the list, and records its parent node	<code>q_new, parent_index</code>	-
<code>get_nearest_neighbor(q_rand)</code>	Finds the node closest to <code>q_rand</code>	<code>q_rand</code>	<code>node, node_idx</code>
<code>get_size()</code>	Returns the number of nodes in the tree	-	<code>size</code>

The `RRT_helpers` class is also provided to assist with collision checks:

Methods	Description	Inputs	Outputs
<code>is_configuration_colliding(phi)</code>	Checks if joint configuration <code>phi</code> collides with any obstacle	<code>joint_angles</code>	True if invalid/collision, False otherwise

Exercise 4: Create an RRT algorithm in the file `obstacle_avoidance_RRT.py`.

Note: Be careful with step sizes, make it too large and you may “skip” through an obstacle.

Test and demonstrate your program across both obstacle setups. When verified, show your working demonstration to the TA.

4.2 Artificial Potential Fields (APF)

The `Potential_Fields_helper` class is provided to evaluate obstacle proximity:

Methods	Description	Inputs	Outputs
<code>get_all_obstacle_distances(self, p4)</code>	Returns a list of dictionaries containing distances and directions from EE to obstacles	<code>p4</code> (EE position in {0} frame)	<code>obstacle_data</code>

Tips:

- Use `J = QArm_Interface.Inv_Jacobian(phi)` to get the inverse of the 4×4 analytic Jacobian.
 - Don’t forget to append a zero to the force vector before multiplying (for the wrist roll degree of freedom that we are ignoring).
- Use `QArm_Interface.write_to_arm(phi)` to move the arm to the joint positions specified by `phi`.
- Make sure to add a slight delay (`time.sleep()`) at the end of your program to allow time for the physical arm to settle into its final configuration.

Exercise 5: In `obstacle_avoidance_potential.py`, implement obstacle avoidance using artificial potential fields. All variables have been assigned a starting value that will be roughly correct; once implemented, do some tuning to improve performance.

Test and demonstrate your program across both obstacle setups. When verified, show your working demonstration to the TA.

LEARNING GOALS

Inverse Kinematics (IK)

- ✓ **Handle Multi-Configuration Solutions:** Update an analytical IK solver to evaluate multiple valid joint configurations and programmatically select the optimal path based on tracking history.
- ✓ **Implement Numerical Solvers:** Build an iterative, Jacobian-based numerical root-finding solver (Newton-Raphson) for joint trajectory generation.
- ✓ **Analyze Convergence & Edge Cases:** Compare geometric vs. numerical methods regarding runtime performance, sensitivity to initial tracking states (ϕ_{prev}), and behavior near workspace boundaries.
- ✓ **Identify Kinematic Singularities:** Observe firsthand how approaching a singularity causes a standard numerical inverse Jacobian to fail or blow up.

Path Planning & Obstacle Avoidance

- ✓ **Build RRT Planners:** Implement a Rapidly-exploring Random Tree algorithm in joint space utilizing custom tree-management and collision-checking classes.
- ✓ **Tune Goal Biasing:** Configure and balance random sampling thresholds vs. goal-directed steps to optimize tree growth speed.
- ✓ **Implement Potential Fields:** Map 3D Cartesian forces acting on an end-effector into joint-space steering torques using the Jacobian Transpose matrix.
- ✓ **Troubleshoot Local Minima:** Identify structural failure modes where competing attractive and repulsive vectors trap the manipulator in dead zones, and propose algorithmic solutions to escape them.



PUSH CHANGES TO GITHUB

5 Post-lab

5.1 Inverse Kinematics Analysis

Exercise 6:

Answer the following comparative questions for **Inverse Kinematics**:

1. Compare the geometric and numerical IK solvers. What are their respective pros and cons? Consider:
 - Did one solver encounter issues or failures where the other did not?
 - In what practical robotics scenarios would you choose an analytical solver over a numerical solver (and vice versa)?
 - How consistent are the solvers in terms of computation runtime and final positioning accuracy?
2. What modifications could improve each method? (For example, how does a Damped Least Squares / Damped Inverse $\dot{\underline{q}} = (\underline{J} + \lambda \underline{I})^{-1} \underline{v}$ mitigate numerical divergence near singularities?)

5.2 Path Planning Analysis

Exercise 7:

Answer the following comparative questions for **Path Planning**:

1. Now that you have implemented both RRT and Artificial Potential Fields, compare their performance and characteristics:
 - How do their planning/execution speeds compare?
 - Can the manipulator get “stuck”? Why? (Explain the local minima trap in potential fields vs. probabilistic completeness in RRT).
 - Assuming you had an accurate map of the environment, how would each method handle an environment with moving or dynamic obstacles?
 - How do they scale to more complex obstacles or higher-DOF manipulators?
 2. What improvements could you make to each method? What are some additions or changes that you can make to address the shortcomings of each method?
-

6 Submission

Make sure to demonstrate your working RRT and Potential Fields scripts to the TA during lab time.

On Canvas, submit a single zip file containing:

- `geometric_IK.py`
- `numerical_IK.py`
- `obstacle_avoidance_RRT.py`
- `obstacle_avoidance_potential.py`
- Completed worksheet
- Individual Reflection (see below)

6.0.1 Individual Reflection

While the codebase is a collaborative group effort, each student must submit a standalone, individual reflection document (maximum 1 page).

1. Specify your group's workflow and the role each member played.
2. Detail at least one specific technical issue or unpredictable behavior your group encountered when running these algorithms. Explain how your group identified and resolved it.
3. Briefly describe everything covered in this lab

Note: To prevent passive participation, TAs will cross-reference these notes against group submissions. Identical or closely paraphrased responses within a group will result in a grade of zero for the individual component.

7 References

- Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python* (3rd ed.). Springer.
- Khatib, O. (1986). “Real-time obstacle avoidance for robot manipulators and mobile robots.” *The International Journal of Robotics Research*, 5(1), 90-98.
- LaValle, S. M. (1998). “Rapidly-exploring random trees: A new tool for path planning.” *Research Report 98-11*, Department of Computer Science, Iowa State University.
- LaValle, S. M., & Kuffner, J. J. (2001). “Rapidly-exploring random trees: Progress and prospects.” *Algorithmic and Computational Robotics: New Directions*, 293-308.
- Whitney, D. E. (1969). “Resolved Motion Rate Control of Manipulators and Human Prostheses.” *IEEE Transactions on Man-Machine Systems*, 10(2), 47-53.